

8. МАКРОСЫ (MACROS)

Макросы, по сути, — это инструмент, заставляющий компилятор писать код за вас. Вы даёте компилятору формулу для генерации кода по некоторым входным параметрам, и компилятор заменяет каждый вызов макроса результатом прохождения через эту формулу. Можно рассматривать макросы как автоматическую подстановку кода, где правила подстановки определяете вы сами.

Макросы в Rust бывают самых разных форм и размеров, что упрощает реализацию многих различных видов генерации кода. Два основных типа — это *декларативные* макросы (declarative macros) и *процедурные* макросы (procedural macros), и мы рассмотрим оба в этой главе. Мы также посмотрим на некоторые способы, которыми макросы могут пригодиться в повседневном программировании, и на некоторые подводные камни, возникающие при более продвинутом использовании.

Программисты, пришедшие из нечестивого мира C и C++, где можно с помощью `#define` менять каждый `true` на `false` или удалять все вхождения ключевого слова `else`, могут ассоциировать макросы с чувством, что делаешь что-то «плохое». Макросы в Rust очень далеки от Дикого Запада макросов C. Они следуют (по большей части) хорошо определённым правилам и довольно устойчивы к неправильному использованию.

Декларативные макросы (Declarative Macros)

Декларативные макросы — это те, что определяются с помощью синтаксиса `macro_rules!`, который позволяет удобно определять функциоподобные макросы без необходимости писать выделенный крейт для этой цели (как приходится делать с процедурными макросами). После того как вы определили декларативный макрос, вы можете вызывать его, используя имя макроса, за которым следует восклицательный знак. Мне нравится думать о таком виде макроса как о своего рода поиске-и-замене с помощью компилятора: он справляется со многими обычными, хорошо структурированными задачами трансформации и устраняет повторяющийся шаблонный код. В вашем опыте работы с Rust вплоть до этого момента большинство макросов, которые вы

распознавали как макросы, скорее всего, были декларативными макросами. Однако обратите внимание, что не все функциоподобные макросы являются декларативными; `macro_rules!` сам по себе — один из примеров этого, а `format_args!` — другой. Суффикс `!` лишь указывает компилятору, что вызов макроса будет заменён на другой исходный код во время компиляции.

ПРИМЕЧАНИЕ Поскольку парсер Rust специально распознаёт и разбирает вызовы макросов, помеченные `!`, использовать их можно только в местах, которые допускает парсер. Они работают в большинстве мест, которые вы ожидаете, например в позиции выражения или внутри блока `impl`, но не везде. Например, на момент написания нельзя вызвать функциоподобный макрос там, где ожидается идентификатор или плечо `match`.

Может быть не сразу очевидно, почему декларативные макросы называются декларативными. В конце концов, разве вы не «декларируете» (объявляете) всё в своей программе? В этом контексте *декларативный* относится к тому факту, что вы не говорите, *как* входные данные макроса должны быть преобразованы в вывод, а лишь то, что вы хотите, чтобы вывод выглядел как *A*, когда вход — *B*. Вы декларируете, что так должно быть, а компилятор разбирается со всей перекоммутацией при разборе, которая должна произойти, чтобы воплотить вашу декларацию в реальность. Это делает декларативные макросы лаконичными и выразительными, хотя также имеет тенденцию делать их довольно загадочными, поскольку у вас есть ограниченный язык, на котором можно выражать свои декларации.

Когда их использовать (When to Use Them)

Декларативные макросы в первую очередь полезны, когда вы обнаруживаете, что пишете один и тот же код снова и снова, и хотели бы, ну, не делать этого. Они лучше всего подходят для довольно механических замен — если вы стремитесь к замысловатым трансформациям кода или большому объёму генерации кода, процедурные макросы, скорее всего, подойдут лучше.

Чаще всего я использую декларативные макросы в случаях, когда обнаруживаю, что пишу повторяющийся и структурно схожий код, например в тестах и реализациях трейтов. Для тестов я часто хочу запустить один и тот же тест несколько раз, но с немного различающимися конфигурациями. У меня может быть что-то вроде того, что показано в листинге 8-1.

```
fn test_inner<T>(init: T, frobnify: bool) { ... }
#[test]
fn test_u8_frobnified() {
    test_inner(u8, true);
}
// ...
#[test]
fn test_i128_not_frobnified() {
    test_inner(i128, false);
}
```

Листинг 8-1: Повторяющийся тестовый код

Хотя это работает, оно слишком многословно, слишком повторяющееся и слишком подвержено ручным ошибкам. С макросами мы можем сделать намного лучше, как показано в листинге 8-2.

```
macro_rules! test_battery {
    ($($t:ty as $name:ident),*) => {
        $(
            mod $name {
                #[test]
                fn frobnified() { test_inner::<$t>(1, true) }
                #[test]
                fn unfrobnified() { test_inner::<$t>(1, false) }
            }
        )*
    }
}
test_battery! {
    u8 as u8_tests,
    // ...
    i128 as i128_tests
};
```

Листинг 8-2: Заставляем макрос повторяться за вас

Этот макрос разворачивает каждую разделённую запятыми директиву в её собственный модуль, который затем содержит два теста: один, вызывающий `test_inner` с `true`, и один — с `false`. Хотя определение макроса нетривиально, оно значительно упрощает добавление новых тестов. Каждый тип — это одна строка в вызове `test_battery!`, а макрос позаботится о генерации тестов как для аргумента `true`, так и для `false`. Мы могли бы также заставить его генерировать тесты для разных значений `init`. Теперь мы значительно снизили вероятность того, что забудем протестировать какую-то конкретную конфигурацию!

История с реализациями трейтов аналогична. Если вы определяете свой собственный трейт, вам часто захочется реализовать этот трейт для ряда типов из стандартной библиотеки, даже если эти реализации тривиальны. Представим, что вы изобрели трейт `Clone` и хотите реализовать его для всех типов `Copy` из стандартной библиотеки. Вместо того чтобы вручную писать реализацию для каждого, вы можете использовать макрос вроде того, что в листинге 8-3.

```
macro_rules! clone_from_copy {
    ($($t:ty),*) => {
        $(impl Clone for $t {
            fn clone(&self) -> Self { *self }
        })*
    }
}
clone_from_copy![bool, f32, f64, u8, i8, /* ... */];
```

Листинг 8-3: Использование макроса для реализации трейта для множества похожих типов одним махом

Здесь мы генерируем реализацию `Clone` для каждого предоставленного типа, тело которой просто использует `*` для копирования из `&self`. Вы можете задаться вопросом, почему мы не добавляем общую (blanket) реализацию `Clone` для `T where T: Copy`. Мы могли бы это сделать, но веская причина не делать этого в том, что это вынудило бы типы в других крейтах также использовать ту же самую реализацию `Clone` для своих собственных типов, которые случайно оказались `Copy`. Экспериментальная возможность компилятора под названием *специализация* (specialization) могла бы предложить обходной путь, но на момент написания стабилизация этой возможности всё ещё довольно далека. Так что пока что нам лучше перечислять типы конкретно. Этот паттерн также выходит за рамки простых пробрасывающих реализаций: например, вы могли бы легко изменить код в листинге 8-3, чтобы реализовать трейт `AddOne` для всех целочисленных типов!

ПРИМЕЧАНИЕ Если вы когда-нибудь задумаетесь, использовать ли обобщённые типы (generics) или декларативный макрос, используйте обобщённые типы. Обобщённые типы, как правило, эргономичнее макросов и гораздо лучше интегрируются с другими конструкциями языка. Рассмотрите такое эмпирическое правило: если ваш код меняется в зависимости от типа, используйте обобщённые типы; в

противном случае используйте макросы.

Как они работают (How They Work)

Каждый язык программирования имеет *грамматику* (grammar), которая диктует, как отдельные символы, составляющие исходный код, могут быть превращены в *токены* (tokens). Токены — это строительные блоки самого нижнего уровня языка, такие как числа, знаки пунктуации, строковые и символьные литералы и идентификаторы; на этом уровне нет различия между ключевыми словами языка и именами переменных. Например, текст `(value + 4)` был бы представлен последовательностью из пяти токенов `(, value, +, 4, )` в Rust-подобной грамматике. Процесс превращения текста в токены также обеспечивает уровень абстракции между остальной частью компилятора и сложными низкоуровневыми деталями разбора текста. Например, в представлении токенов нет понятия пробельных символов, а `/*"foo"*/` и `"/*foo*/"` имеют различные представления (первое не является токеном, а второе — строковый литеральный токен с содержимым `/*foo*/`).

После того как исходный код превращён в последовательность токенов, компилятор проходит по этой последовательности и присваивает токенам синтаксическое значение. Например, токены `()` составляют группу, токены `!` обозначают вызовы макросов и так далее. Это процесс *разбора* (parsing), который в конечном итоге производит абстрактное синтаксическое дерево (abstract syntax tree, AST), описывающее структуру, представленную исходным кодом. В качестве примера рассмотрим выражение `let x = || 4`, которое состоит из последовательности токенов `let` (ключевое слово), `x` (идентификатор), `=` (пунктуация), двух экземпляров `|` (пунктуация) и `4` (литерал). Когда компилятор превращает это в синтаксическое дерево, оно представляется как *оператор* (statement), чей *паттерн* — это *идентификатор* `x`, а чьё правостороннее *выражение* — это *замыкание* (closure) с пустым *списком аргументов* и *литеральным выражением целочисленного литерала* `4` в качестве тела. Обратите внимание, насколько представление синтаксического дерева богаче последовательности токенов, поскольку оно присваивает синтаксическое значение комбинациям токенов, следующим из грамматики языка.

Макросы Rust диктуют, в какое синтаксическое дерево превращается заданная последовательность токенов — когда компилятор встречает вызов

макроса во время разбора, ему нужно вычислить макрос, чтобы определить токены замены, которые в конечном итоге станут синтаксическим деревом для вызова макроса. Однако в этот момент компилятор всё ещё разбирает токены и может быть не в состоянии вычислить макрос пока, поскольку он лишь разобрал токены определения макроса. Поэтому вместо этого компилятор откладывает разбор всего, что содержится внутри разделителей вызова макроса, и запоминает входную последовательность токенов. Когда компилятор готов вычислить указанный макрос, он вычисляет макрос над последовательностью токенов, разбирает выдаваемые им токены и подставляет получившееся синтаксическое дерево в дерево там, где был вызов макроса.

Технически компилятор всё же делает небольшой разбор входных данных для макроса. В частности, он разбирает базовые вещи вроде строковых литералов и групп с разделителями и таким образом производит последовательность *деревьев* токенов (token trees), а не просто токенов. Например, код `x - (a.b + 4)` разбирается как последовательность из трёх деревьев токенов. Первое дерево токенов — это одиночный токен — идентификатор `x`, второе — одиночный токен — знак пунктуации `-`, а третье — это группа (использующая круглые скобки в качестве разделителя), которая сама состоит из последовательности из пяти деревьев токенов: `a` (идентификатор), `.` (пунктуация), `b` (другой идентификатор), `+` (другой токен пунктуации) и `4` (литерал). Это означает, что вход для макроса не обязательно должен быть валидным Rust, но должен состоять из кода, который компилятор Rust может разобрать. Например, вы не могли бы написать `for <- x` в Rust вне вызова макроса, но внутри вызова макроса можете, пока макрос производит валидный синтаксис. С другой стороны, вы не можете передать макросу `for {`, потому что у него нет закрывающей скобки.

Декларативные макросы всегда генерируют валидный Rust в качестве вывода. Вы не можете заставить макрос генерировать, скажем, первую половину вызова функции или `if` без блока, следующего за ним. Декларативный макрос должен генерировать выражение (по сути, что-либо, что можно присвоить переменной), оператор вроде `let x = 1;`, элемент (item) вроде определения трейта или блока `impl`, тип или паттерн `match`. Это делает макросы Rust устойчивыми к неправильному использованию: вы просто не можете написать декларативный макрос, который генерирует невалидный код Rust, потому что само определение макроса не скомпилируется!

Вот, собственно, и всё о декларативных макросах на высоком уровне — когда компилятор встречает вызов макроса, он передаёт макросу токены, содержащиеся внутри разделителей вызова, разбирает получившийся поток токенов и заменяет вызов макроса получившимся AST.

Как писать декларативные макросы (How to Write Declarative Macros)

Исчерпывающее объяснение всего синтаксиса, который поддерживают декларативные макросы, выходит за рамки этой книги. Однако мы рассмотрим основы, поскольку есть некоторые странности, на которые стоит указать.

Декларативные макросы состоят из двух главных частей: *сопоставителей* (matchers) и *транскрайберов* (transcribers). Данный макрос может иметь много сопоставителей, и у каждого сопоставителя есть связанный транскрайбер. Когда компилятор находит вызов макроса, он проходит по сопоставителям макроса от первого до последнего, и когда он находит сопоставитель, который сопоставляется с токенами в вызове, он заменяет вызов, проходя по токенам соответствующего транскрайбера. Листинг 8-4 показывает, как соединяются разные части правила декларативного макроса.

```
macro_rules! /* имя макроса */ {  
    /* 1-й сопоставитель */ => { /* 1-й транскрайбер */ };  
    /* 2-й сопоставитель */ => { /* 2-й транскрайбер */ };  
}
```

Листинг 8-4: Компоненты определения декларативного макроса

Сопоставители (Matchers)

Можно представлять сопоставитель макроса как дерево токенов, которое компилятор пытается крутить и сгибать заранее определёнными способами, чтобы сопоставить его с входным деревом токенов, которое ему было дано в месте вызова. В качестве примера рассмотрим макрос с сопоставителем `$a:ident + $b:expr`. Этот сопоставитель найдёт совпадение с любым идентификатором (`:ident`), за которым следует знак плюс, за которым следует любое выражение Rust (`:expr`). Если макрос вызывается с `x + 3 * 5`, компилятор замечает, что сопоставитель совпадает, если он устанавливает `$a = x` и `$b = 3 * 5`. Хотя `*` никогда не появляется в сопоставителе, компилятор понимает, что `3 * 5` — это валидное выражение и что поэтому оно может быть сопоставлено с

`$b:expr`, который принимает что угодно как выражение (часть `:expr`).

Сопоставители могут становиться довольно запутанными, но они обладают огромной выразительной силой, во многом как регулярные выражения. В качестве не слишком запутанного примера этот сопоставитель принимает последовательность `$(())` из одной или более (+) разделённых запятыми (,) пар ключ/значение, заданных в формате `key => value`:

```
$(key:expr => value:expr),+
```

И, что особенно важно, код, который вызывает макрос с этим сопоставителем, может дать произвольно сложное выражение для ключа или значения — магия сопоставителей обеспечит, что выражения ключа и значения будут разделены соответствующим образом.

Правила макросов поддерживают широкий набор *типов фрагментов* (fragment types); вы уже видели `:ident` для идентификаторов и `:expr` для выражений, но есть также `:ty` для типов и даже `:tt` для любого одиночного дерева токенов! Полный список типов фрагментов можно найти в главе 3 справочника по языку Rust (<https://doc.rust-lang.org/reference/macros-by-example.html>). Они, плюс механизм для повторного сопоставления паттерна `$(())`, позволяют сопоставлять большинство простых паттернов кода. Если же вы обнаружите, что трудно выразить нужный вам паттерн с помощью сопоставителя, возможно, вам захочется попробовать процедурный макрос, где вам не нужно следовать строгому синтаксису, которого требует `macro_rules!`. Мы рассмотрим их подробнее позже в этой главе.

Транскрайберы (Transcribers)

После того как компилятор сопоставил сопоставитель декларативного макроса, он генерирует код, используя связанный с сопоставителем транскрайбер. Переменные, определённые сопоставителем макроса, называются *метапеременными* (metavariables), и компилятор подставляет вместо каждого вхождения каждой метапеременной в транскрайбере (например, `$key` в примере из предыдущего раздела) входные данные, которые совпали с этой частью сопоставителя. Если в сопоставителе есть повторение (например, `$(()),+` в том же примере), вы можете использовать тот же синтаксис в транскрайбере, и он будет повторён один раз для каждого совпадения во входных данных, причём каждое разворачивание содержит соответствующую

подстановку для каждой метапеременной на этой итерации. Например, для сопоставителя `$key/$value` мы могли бы написать следующий транскрайбер, чтобы генерировать вызов `insert` в какую-то карту (`map`) для каждой совпавшей пары `$key/$value`:

```
$(map.insert($key, $value);)+
```

Обратите внимание, что здесь мы хотим точку с запятой для каждого повторения, а не просто для разделения повторения, поэтому мы помещаем точку с запятой внутрь круглых скобок повторения.

ПРИМЕЧАНИЕ Вы должны использовать метапеременную в каждом повторении в транскрайбере, чтобы компилятор знал, какое повторение в сопоставителе использовать (в случае, если их больше одного).

Гигиена (Hygiene)

Вы, возможно, слышали, что макросы Rust *гигиеничны* (hygienic), и, возможно, что эта гигиеничность делает их более безопасными или приятными в работе, без обязательного понимания, что это значит. Когда мы говорим, что макросы Rust гигиеничны, мы имеем в виду, что декларативный макрос (как правило) не может влиять на переменные, которые ему не были явно переданы. Тривиальный пример: если вы объявляете переменную с именем `foo`, а затем вызываете макрос, который также определяет переменную с именем `foo`, то `foo` макроса по умолчанию не виден в месте вызова (место, откуда вызывается макрос). Аналогично, макросы не могут получить доступ к переменным, определённым в месте вызова (даже `self`), если они не переданы явно.

Большую часть времени можно представлять идентификаторы макроса как существующие в их собственном пространстве имён, отдельном от остального кода, в который они разворачиваются. В качестве примера взгляните на код в листинге 8-5, в котором есть макрос, пытающийся (и не способный) затенить переменную в месте вызова.

```
macro_rules! let_foo {  
  ($x:expr) => {  
    let foo = $x;
```

```
    }  
  }  
  let foo = 1;  
  // разворачивается в let foo = 2;  
  let_foo!(2);  
  assert_eq!(foo, 1);
```

Листинг 8-5: Макросы существуют в своих собственных маленьких вселенных. По большей части.

После того как компилятор разворачивает `let_foo!(2)`, выглядит так, будто `assert` должен провалиться. Однако `foo` из исходного кода и `foo`, сгенерированный макросом, существуют в разных вселенных и не имеют отношения друг к другу, кроме того что они случайно разделяют человекочитаемое имя. На самом деле компилятор пожалуется, что `let foo` в макросе — это неиспользуемая переменная. Эта гигиена очень помогает делать макросы проще для отладки — вам не нужно беспокоиться о случайном затенении или перезаписи переменных у вызывающего макрос только потому, что вы случайно выбрали те же имена переменных!

Это гигиеничное разделение, однако, не распространяется за пределы идентификаторов переменных. Декларативные макросы действительно разделяют пространство имён для типов, модулей и функций с местом вызова. Это означает, что ваш макрос может определить новые функции, которые можно вызвать в вызывающей области, добавить новые реализации к типу, определённого в другом месте (и не переданному), ввести новый модуль, к которому затем можно обращаться там, где был вызван макрос, и так далее. Это сделано преднамеренно — если бы макросы не могли влиять на более широкий код подобным образом, было бы гораздо обременительнее использовать их для генерации типов, реализаций трейтов и функций, а именно в этом они наиболее полезны.

Отсутствие гигиены для типов в макросах особенно важно, когда вы пишете макрос, который хотите экспортировать из вашего крейта. Чтобы макрос был действительно переиспользуемым, нельзя предполагать ничего о том, какие типы будут в области видимости у вызывающего. Может быть, код, вызывающий ваш макрос, имеет определённый `mod std {}` или импортировал свой собственный тип `Result`. Чтобы быть в безопасности, убедитесь, что вы используете полностью определённые типы вроде `::core::option::Option` или `::alloc::boxed::Box`. Если вам конкретно нужно сослаться на что-то в крейте,

который определяет макрос, используйте специальную метаварiable `$crate`.

ПРИМЕЧАНИЕ Избегайте использования путей `::std`, если можете, чтобы макрос продолжал работать в крейтах `no_std`.

Вы можете выбрать разделение идентификаторов между макросом и его вызывающим, если хотите, чтобы макрос влиял на конкретную переменную в области видимости вызывающего. Ключ в том, чтобы помнить, где возник идентификатор, потому что именно с этим пространством имён будет связан идентификатор. Если вы помещаете `let foo = 1;` в макрос, идентификатор `foo` происходит из макроса и никогда не будет доступен пространству имён идентификаторов вызывающего. Если же, с другой стороны, макрос принимает `$foo:ident` в качестве аргумента, а затем пишет `let $foo = 1;`, то когда вызывающий вызывает макрос с `!(foo)`, идентификатор будет происходить у вызывающего и поэтому будет ссылаться на `foo` в области видимости вызывающего.

Идентификатор не обязательно передавать столь явно; любой идентификатор, который появляется в коде, происходящем за пределами макроса, будет ссылаться на идентификатор в области видимости вызывающего. В примере в листинге 8-6 идентификатор переменной появляется внутри `:expr`, но тем не менее имеет доступ к переменной в области видимости вызывающего.

```
macro_rules! please_set {
  ($i:ident, $x:expr) => {
    $i = $x;
  }
}
let mut x = 1;
please_set!(x, x + 1);
assert_eq!(x, 2);
```

Листинг 8-6: Предоставление макросам доступа к идентификаторам в месте вызова

Мы могли бы использовать `= $i + 1` в макросе вместо этого, но мы не могли бы использовать `= x + 1`, поскольку имя `x` недоступно в области определения макроса.

Последнее замечание о декларативных макросах и областях видимости: в

отличие практически от всего остального в Rust, декларативные макросы существуют в исходном коде только после того, как они объявлены. Если вы попытаетесь использовать макрос, который вы определяете далее в файле, это не сработает! Это применяется глобально к вашему проекту; если вы объявляете макрос в одном модуле и хотите использовать его в другом, модуль, в котором вы объявляете макрос, должен появиться в крейте раньше, а не позже. Если `foo` и `bar` — это модули в корне крейта, и `foo` объявляет макрос, который хочет использовать `bar`, то `mod foo` должен появиться перед `mod bar` в `lib.rs`!

ПРИМЕЧАНИЕ Есть одно исключение из этой странной области видимости макросов (формально называемой *текстовой областью видимости* (textual scoping)), и это если вы пометите макрос с помощью `#[macro_export]`. Эта аннотация фактически поднимает макрос в корень крейта и помечает его как `pub`, так что его можно использовать в любом месте вашего крейта или зависимыми от вашего крейта.

Процедурные макросы (Procedural Macros)

Можно представлять процедурный макрос как комбинацию парсера и генерации кода, где вы пишете связующий код между ними. На высоком уровне с процедурными макросами компилятор собирает последовательность входных токенов для макроса и запускает вашу программу, чтобы определить, какими токенами их заменить.

Процедурные макросы называются так потому, что вы определяете, *как* генерировать код по некоторым входным токенам, а не просто пишете, какой код получается на выходе. На стороне компилятора задействовано очень мало интеллекта — насколько ему известно, процедурный макрос — это более или менее препроцессор исходного кода, который может заменять код произвольным образом. Требование, чтобы ваш вход можно было разобрать как поток токенов Rust, по-прежнему сохраняется, но на этом всё!

Типы процедурных макросов (Types of Procedural Macros)

Процедурные макросы бывают трёх разных вкусов, каждый из которых специализирован для конкретного распространённого случая использования:

- Функциоподобные макросы (function-like), как те, что генерирует `macro_rules!`
- Атрибутные макросы (attribute macros), как `#[test]`
- Производные макросы (derive macros), как `#[derive(Serialize)]`

Все три типа используют один и тот же базовый механизм: компилятор предоставляет вашему макросу последовательность токенов и ожидает, что вы произведёте в ответ последовательность токенов, которые (вероятно) связаны с входным деревом. Однако они различаются тем, как макрос вызывается и как обрабатывается его вывод. Мы кратко рассмотрим каждый из них.

Функциоподобные макросы (Function-Like Macros)

Функциоподобный макрос — это простейшая форма процедурного макроса. Как и декларативный макрос, он просто заменяет код макроса в месте вызова на код, который возвращает процедурный макрос. Однако, в отличие от декларативных макросов, все защитные ограждения сняты: эти макросы (как и все процедурные макросы) не обязаны быть гигиеничными и не защитят вас от взаимодействия с идентификаторами в окружающем коде в месте вызова. Вместо этого ожидается, что ваши макросы будут явно указывать, какие идентификаторы должны пересекаться с окружающим кодом (используя `Span::call_site`), а какие должны рассматриваться как приватные для макроса (используя `Span::mixed_site`, который мы обсудим позже).

Атрибутные макросы (Attribute Macros)

Атрибутный макрос также целиком заменяет элемент, которому присвоен атрибут, но он принимает два входа: дерево токенов, которое появляется в атрибуте (за вычетом имени атрибута), и дерево токенов всего элемента, к которому он прикреплён, включая любые другие атрибуты, которые может иметь этот элемент. Атрибутные макросы позволяют легко написать процедурный макрос, который преобразует элемент, например добавляя пролог или эпилог к определению функции (как делает `#[test]`) или изменяя поля структуры.

Производные макросы (Derive Macros)

Производный макрос немного отличается от двух других тем, что он *добавляет* к цели макроса, а не заменяет её. Хотя это ограничение может

показаться серьёзным, производные макросы были одним из изначальных мотивирующих факторов создания процедурных макросов. В частности, крейту `serde` нужны были производные макросы, чтобы реализовать свою теперь хорошо известную магию `#[derive(Serialize, Deserialize)]`.

Производные макросы, можно сказать, простейшие из процедурных макросов, поскольку у них такая жёсткая форма: вы можете добавлять элементы только после аннотированного элемента; вы не можете заменить аннотированный элемент и не можете принимать аргументы при выведении. Производные макросы всё же позволяют вам определять *вспомогательные атрибуты* (`helper attributes`) — атрибуты, которые можно размещать внутри аннотированного типа, чтобы дать подсказки производному макросу (как `#[serde(skip)]`), — но они функционируют в основном как маркеры и не являются независимыми макросами.

Цена процедурных макросов (The Cost of Procedural Macros)

Прежде чем мы поговорим о том, когда каждый из различных типов процедурных макросов уместен, стоит обсудить, почему вы можете захотеть дважды подумать, прежде чем тянуться к процедурному макросу, а именно — увеличение времени компиляции.

Процедурные макросы могут значительно увеличить время компиляции по двум главным причинам. Первая в том, что они склонны приносить с собой довольно тяжёлые зависимости. Например, крейт `syn`, который предоставляет парсер для потоков токенов Rust, делающий написание процедурных макросов гораздо проще, может компилироваться десятки секунд при включённых всех возможностях. Вы можете (и должны) смягчить это, отключив возможности, которые вам не нужны, и компилируя ваши процедурные макросы в отладочном режиме (`debug mode`), а не в режиме выпуска (`release mode`). Код часто компилируется в несколько раз быстрее в отладочном режиме, и для большинства процедурных макросов вы даже не заметите разницы во времени выполнения.

Вторая причина, по которой процедурные макросы увеличивают время компиляции, в том, что они облегчают генерацию большого количества кода так, что вы этого даже не осознаёте. Хотя макрос избавляет вас от необходимости фактически набирать сгенерированный код, он не избавляет компилятор от необходимости разбирать, компилировать и оптимизировать

его. По мере того как вы используете больше процедурных макросов, этот сгенерированный шаблонный код накапливается и может раздуть ваше время компиляции.

При этом фактическое время выполнения процедурных макросов редко является фактором в общем времени компиляции. Хотя компилятор должен ждать, пока процедурный макрос сделает своё дело, прежде чем он сможет продолжить, на практике большинство процедурных макросов не выполняют никаких тяжёлых вычислений. При этом если ваш процедурный макрос особенно сложный, может оказаться, что ваши компиляции тратят значительную часть времени выполнения на код вашего процедурного макроса, за чем стоит следить!

Итак, вы думаете, что хотите макрос (So You Think You Want a Macro)

Давайте теперь рассмотрим некоторые хорошие способы использования каждого типа процедурного макроса. Начнём с простого: производных макросов.

Когда использовать производные макросы (When to Use Derive Macros)

Производные макросы используются для одной вещи и только одной: автоматизировать реализацию трейта там, где автоматизация возможна. Не у всех трейтов есть очевидные автоматизированные реализации, но у многих есть. На практике стоит рассматривать добавление производного макроса для трейта только в том случае, если трейт реализуется часто и если его реализация для любого данного типа довольно очевидна. Первое из этих условий может показаться здравым смыслом; если ваш трейт будет реализован только один-два раза, вероятно, не стоит писать и поддерживать запутанный производный макрос для него.

Второе условие может показаться более странным, однако: что значит, чтобы реализация была «очевидной»? Рассмотрим трейт вроде `Debug`. Если бы вам сказали, что делает `Debug`, и показали тип, вы бы, вероятно, ожидали, что реализация `Debug` выведет имя каждого поля рядом со значением этого поля. И именно это делает `derive(Debug)`. А что насчёт `Clone`? Вы бы, вероятно, ожидали, что он просто клонирует каждое поле, — и снова, именно это делает

`derive(Clone)`. С `derive(serde::Serialize)` мы ожидаем, что он сериализует каждое поле и его значение, и он делает именно это. В целом вы хотите, чтобы выводение трейта соответствовало интуиции разработчика о том, что оно, вероятно, делает. Если для трейта нет очевидного вывода или, что ещё хуже, если ваше выводение не соответствует очевидной реализации, то вам, вероятно, лучше не давать ему производный макрос.

Когда использовать функциоподобные макросы (When to Use Function-Like Macros)

Для функциоподобных макросов труднее дать общее эмпирическое правило. Можно сказать, что вам следует использовать функциоподобные макросы, когда вы хотите функциоподобный макрос, но не можете выразить его с помощью `macro_rules!`, но это довольно субъективное руководство. С декларативными макросами можно сделать многое, если действительно постараться, в конце концов!

Есть две особенно веские причины тянуться к функциоподобному макросу:

- Если у вас уже есть декларативный макрос, и его определение становится настолько запутанным, что макрос трудно поддерживать.
- Если у вас есть чистая функция, которую вам нужно иметь возможность выполнять во время компиляции, но которую вы не можете выразить с помощью `const fn`. Пример этого — крейт `phf`, который генерирует карту хешей (hash map) с использованием идеальной хеш-функции (perfect hash function) по заданному во время компиляции набору ключей. Другой — `hex-literal`, который принимает строку шестнадцатеричных символов и заменяет её соответствующими байтами. В целом всё, что не просто преобразует вход во время компиляции, а фактически вычисляет над ним, вероятно, будет хорошим кандидатом.

Я не рекомендую тянуться к функциоподобному макросу только для того, чтобы можно было нарушить гигиену внутри вашего макроса. Гигиена для функциоподобных макросов — это возможность, которая избегает многих головных болей при отладке, и вам следует очень тщательно подумать, прежде чем намеренно нарушать её.

Когда использовать атрибутные макросы (When to Use Attribute Macros)

Остаются атрибутные макросы. Хотя они, можно сказать, самые общие из процедурных макросов, они также труднее всего поддаются определению, когда их использовать. За годы, снова и снова, я видел четыре способа, которыми атрибутные макросы добавляют огромную ценность.

Генерация тестов (Test generation)

Очень часто хочется запустить один и тот же тест при множестве различных конфигураций или много похожих тестов с одним и тем же кодом начальной инициализации (bootstrapping). Хотя декларативный макрос может позволить вам выразить это, ваш код часто проще читать и поддерживать, если у вас есть атрибут вроде `#[foo_test]`, который вводит установочный пролог и эпилог в каждый аннотированный тест, или повторяемый атрибут вроде `#[test_case(1)]` `#[test_case(2)]`, чтобы пометить, что данный тест должен повторяться несколько раз, по одному разу с каждым входом.

Аннотации фреймворков (Framework annotations)

Библиотеки вроде `rocket` используют атрибутные макросы для дополнения функций и типов дополнительной информацией, которую затем использует фреймворк, без того чтобы пользователю пришлось делать много ручной конфигурации. Гораздо удобнее иметь возможность написать `#[get("/<name>")]` `fn hello(name: String)`, чем настраивать конфигурационную структуру с указателями на функции и тому подобным. По сути, атрибуты составляют миниатюрный предметно-ориентированный язык (domain-specific language, DSL), который скрывает массу шаблонного кода, который иначе был бы необходим. Аналогично, фреймворк асинхронного ввода-вывода `tokio` позволяет использовать `#[tokio::main] async fn main()`, чтобы автоматически настроить среду выполнения (runtime) и запустить ваш асинхронный код, тем самым избавляя вас от написания одного и того же установочного кода времени выполнения в функции `main` каждого асинхронного приложения.

Прозрачное промежуточное ПО (Transparent middleware)

Некоторые библиотеки хотят внедряться в ваше приложение ненавязчивыми способами, чтобы предоставить добавленную ценность, которая не меняет функциональность приложения. Например, библиотеки трассировки и

логирования вроде `tracing` и библиотеки сбора метрик вроде `metered` позволяют прозрачно инструментировать функцию, добавляя к ней атрибут, а затем каждый вызов этой функции будет выполнять некоторый дополнительный код, продиктованный библиотекой.

Преобразователи типов (Type transformers)

Иногда вы хотите выйти за рамки простого выведения трейтов для типа и фактически изменить определение типа каким-то фундаментальным образом. В этих случаях атрибутные макросы — это путь. Крейт `pin_project` — отличный пример этого: его основная цель не реализовать какой-то конкретный трейт, а скорее обеспечить, чтобы любой закреплённый (`pinned`) доступ к полям данного типа происходил согласно строгим правилам, изложенным типом `Pin` Rust и трейтом `Unpin` (мы поговорим больше об этих типах в главе 9). Он делает это, генерируя дополнительные вспомогательные типы, добавляя методы к аннотированному типу и вводя статические проверки безопасности, чтобы пользователи случайно не выстрелили себе в ногу. Хотя `pin_project` мог бы быть реализован с помощью процедурного производного макроса, та производная реализация трейта, скорее всего, не была бы очевидной, что нарушает одно из наших правил о том, когда использовать процедурные макросы.

Как они работают? (How Do They Work?)

В сердце всех процедурных макросов находится тип `TokenStream`, по которому можно итерироваться, чтобы получить отдельные элементы `TokenTree`, составляющие этот поток токенов. `TokenTree` — это либо одиночный токен (как идентификатор, знак пунктуации или литерал), либо другой `TokenStream`, заключённый в разделитель вроде `()` или `{}`. Проходя по `TokenStream`, вы можете разобрать какой угодно синтаксис, пока отдельные токены являются валидными токенами Rust. Если вы хотите разобрать ваш вход именно как код Rust, вам, скорее всего, захочется использовать крейт `syn`, который реализует полный парсер Rust и может превратить `TokenStream` в легко обходимое `AST` Rust.

С большинством процедурных макросов вы хотите не только разобрать `TokenStream`, но и произвести код Rust, который будет внедрён в программу, вызывающую процедурный макрос. Есть два главных способа сделать это. Первый — вручную сконструировать `TokenStream` и расширять его по одному

TokenTree за раз. Второй — использовать реализацию FromStr у TokenStream, которая позволяет вам разобрать строку, содержащую код Rust, в TokenStream с помощью `".parse::<TokenStream>()`. Вы можете также смешивать и сочетать их; если вы хотите добавить какой-то код перед входом вашего макроса, просто сконструируйте TokenStream для пролога, а затем используйте трейт Extend, чтобы дописать исходный вход.

ПРИМЕЧАНИЕ TokenStream также реализует Display, который красиво печатает токены в потоке. Это очень удобно для отладки!

Токены чуть-чуть более волшебны, чем я описывал до сих пор, в том смысле, что каждый токен, и действительно каждый TokenTree, также имеет *диапазон* (span). Диапазоны — это то, как компилятор связывает сгенерированный код обратно с исходным кодом, который его породил. Диапазон каждого токена отмечает, где этот токен возник. Например, рассмотрим (декларативный) макрос вроде того, что в листинге 8-7, который генерирует тривиальную реализацию Debug для предоставленного типа.

```
macro_rules! name_as_debug {
    ($t:ty) => {
        impl ::core::fmt::Debug for $t {
            fn fmt(&self, f: &mut ::core::fmt::Formatter<'_>) -> ::core::fmt::Result
            { ::core::write!(f, ::core::stringify!($t)) }
        }
    };
}
```

Листинг 8-7: Очень простой макрос для реализации Debug

Теперь представим, что кто-то вызывает этот макрос с `name_as_debug!(u31)`. Технически ошибка компилятора возникает внутри макроса, конкретно там, где мы пишем `for $t` (другое использование `$t` может обработать невалидный тип). Но нам бы хотелось, чтобы компилятор указал пользователю на `u31` в его коде — и действительно, именно для этого диапазоны нам и нужны.

Диапазон `$t` в сгенерированном коде — это код, отображённый на `$t` в вызове макроса. Эта информация затем переносится через компилятор и связывается с возможной ошибкой компилятора. Когда эта ошибка компилятора в конце концов печатается, компилятор напечатает ошибку изнутри макроса, говоря, что тип `u31` не существует, но подсветит аргумент `u31` в вызове макроса, поскольку это связанный с ошибкой диапазон!

Диапазоны довольно гибки, и они позволяют писать процедурные макросы,

которые могут производить сложные сообщения об ошибках, если вы используете макрос `compile_error!`. Как следует из его названия, `compile_error!` заставляет компилятор выдать ошибку везде, где он размещён, с предоставленной строкой в качестве сообщения. Это может показаться не очень полезным, пока вы не сочетаете его с диапазоном. Устанавливая диапазон `TokenTree`, который вы генерируете для вызова `compile_error!`, равным диапазону какого-то подмножества входных данных, вы фактически говорите компилятору выдать эту ошибку компилятора и указать пользователю на эту часть исходного кода. Вместе эти два механизма позволяют макросу производить ошибки, которые кажутся происходящими из соответствующей части кода, хотя фактическая ошибка компилятора находится где-то в сгенерированном коде, который пользователь даже не видит!

ПРИМЕЧАНИЕ Если вам когда-нибудь было любопытно, как работает обработка ошибок в `syn`, его тип `Error` реализует метод `Error::to_compile_error`, который превращает его в `TokenStream`, содержащий только директиву `compile_error!`. Что особенно изящно в типе `Error` из `syn`, так это то, что внутри он содержит коллекцию ошибок, каждая из которых производит отдельную директиву `compile_error!` со своим собственным диапазоном, так что вы можете легко производить несколько независимых ошибок из вашего процедурного макроса.

Сила диапазонов на этом не заканчивается; диапазоны — это также то, как реализована гигиена макросов Rust. Когда вы конструируете токен `Ident`, вы также даёте диапазон для этого идентификатора, и этот диапазон диктует область видимости этого идентификатора. Если вы устанавливаете диапазон идентификатора равным `Span::call_site()`, идентификатор разрешается там, откуда был вызван макрос, и таким образом не будет изолирован от окружающей области видимости. Если же, с другой стороны, вы устанавливаете его равным `Span::mixed_site()`, то (переменные) идентификаторы разрешаются в месте определения макроса, и таким образом будут полностью гигиеничны по отношению к одноимённым переменным в месте вызова. `Span::mixed_site` называется так потому, что он соответствует правилам гигиены идентификаторов для `macro_rules!`, которые, как мы обсуждали ранее, «смешивают» разрешение идентификаторов между использованием места определения макроса для переменных и использованием места вызова для типов, модулей и всего остального.

Итоги (Summary)

В этой главе мы рассмотрели как декларативные, так и процедурные макросы и посмотрели на то, когда каждый из них может быть полезен в вашем собственном коде. Мы также глубже погрузились в механизмы, лежащие в основе каждого типа макроса, и в некоторые возможности и подводные камни, о которых стоит знать, когда вы пишете свои собственные макросы. В следующей главе мы начнём наше путешествие в асинхронное программирование и трейт `Future`. Обещаю — это всего лишь на следующей странице.